

IES: AgriSense - Product specification report

Gonçalo Fonseca [118920], Marta Condeço [112903], Diego Aguilar [117122], Tiago Coelho [118745]

v2026-05-26

1 Introduction	2
1.1 Overview of the project	2
1.2 Project limitation	2
2 Product concept and requirements	2
2.1 Vision statement	2
2.2 Personas and scenarios	3
3 Architecture notebook	4
3.1 Key quality requirements	4
Reliability & Availability	4
Usability	4
3.2 Architecture view	5
3.3 Technology Stack	5
4 Information Model	6
4.1 Database Entities	6
4.2 API Documentation	8
4.3 Module Interactions	9
5 Data Simulation	9
5.1 Architecture Overview	10
5.2 Base Module (base.py)	10
5.3 Sensor Simulators	10
5.4 Message Format	11
5.5 Deployment	11
6 Observability (ELK)	11
6.1 Architecture	11
6.2 Dashboards	12
7 Deployment	14
8 Future Implementations and Enhancements	14
8.1 Production Deployment Requirements	14
8.1.1 Environment Configuration and Secrets Management	14
8.1.2 Data Retention and TimescaleDB Policies	15
8.1.3 Container Orchestration	15
8.2 Functional Enhancements	15
8.2.1 Automated Alerting	15
8.2.2 Real Sensor Integration	15

8.2.3 Live Metric Streaming	15
8.2.4 Role-Based Access Control Refinement	16
8.2.5 Audit Logging	16
8.2.6 Mobile Application	16

1 Introduction

1.1 Overview of the project

This project was developed within the scope of the Introduction to Software Engineering curricular unit and focuses on the creation of a modern, user-centric dashboard platform for farm management.

The platform is designed to provide **fast, centralized access to multiple dashboards**, each presenting critical information about the status and performance of farm animals.

The system aggregates key metrics, such as:

- Health indicators
- Activity levels
- Environmental conditions

This enables users to monitor their livestock efficiently and make informed decisions in real time.

With an emphasis on clarity, responsiveness, and ease of use, the platform aims to simplify complex data into intuitive visual insights, ultimately supporting better farm management and improving overall operational awareness.

1.2 Project limitation

As this project is a proof of concept and does not incorporate real actuators, it operates within a simulated environment rather than relying on real-world data. While the simulation is designed to closely mimic realistic conditions, the results may not fully reflect the behavior and variability of a real-world deployment.

2 Product concept and requirements

2.1 Vision statement

The system addresses the growing need for advanced animal tracking solutions that enable farmers to monitor livestock in real time and respond rapidly to potential issues. It provides a comprehensive platform for managing multiple barns within a farm, each housing different animal species.

Within each barn, the system continuously monitors individual animals by collecting key health and activity metrics, including body temperature, movement, heart rate, location, respiratory rate, and posture (e.g., standing or resting). In addition, environmental conditions are tracked to provide a complete view of the animals' surroundings.

By combining real-time data collection with intuitive visualization, the platform empowers farmers to make informed decisions, improve animal welfare, and enhance overall farm efficiency.

2.2 Personas and scenarios

Persona 1: Farmer

Motivation: To easily monitor the vital signs and overall well-being of livestock in real time, enabling faster and more informed decision-making.

Scenario: “It’s a Sunday morning after several particularly cold days, and I’m concerned about the well-being of my animals, even though I check on them daily. I want a quick and reliable way to assess their condition without having to inspect each one manually. I open the AgriSense app and immediately access the dashboard overview. I notice that one of my animals is showing abnormal vital signs. Recognizing the urgency, I quickly contacted my veterinarian through the app, which automatically shares the animal’s current status and relevant data.”

User Stories:

1. As a **Farmer**, I want to add a barn to my farm so that I can accurately represent my farm in the system.
2. As a **Farmer**, I want to remove a barn from my farm so that I can keep my farm structure accurate and up to date when a barn is no longer in use.
3. As a **Farmer**, I want to add an animal to a barn so that I can keep my barn up to date when a new animal arrives.
4. As a **Farmer**, I want to remove an animal from a barn so that I can keep my barn up to date when an animal is no longer alive.
5. As a **Farmer**, I want to view a barn's dashboard so that I can keep up with the current status of the barn.
6. As a **Farmer**, I want to view a barn's animal list so that I can see what animals are in that barn.
7. As a **Farmer**, I want to view an animal's dashboard so that I can see its vital signs (heart rate, temperature, stress, movement, weight).
8. As a **Farmer**, I want to view critical notifications when an animal exhibits abnormal signs so that I can respond quickly and avoid a more serious situation.
9. As a **Farmer**, I want to write notes on an animal's profile so that I can record observations for future reference or for the vet.
10. As a **Farmer**, I want to notify the vet by email with a PDF health report so that the vet has all the information needed to act.

Persona 2: Manager

Motivation: Maintain a clean, up-to-date registry of farms and farmers. Prevent unauthorised access.

Scenario: “Maria is a system manager at AgriSense. After being contacted by João, a new farmer who wants to monitor his livestock, she logs in and navigates to the Farmer List, where she creates his account by filling in his name, email, and a temporary password. The system registers João in Keycloak with the farmer role. She then creates a farm called “Quinta do João” in Aveiro and

associates it with João's account. Maria shares the credentials with João, who can now log in and manage his barns and animals. When a different farmer later retires, Maria finds their account in the Farmer List, deletes it, and the farmer immediately loses access to the system.”

User Storie:

1. As a **Manager**, I want to register a farmer so that new farmers can access the system and manage their farms.
2. As a **Manager**, I want to delete a farmer's account so that inactive or unauthorised users no longer have access to the system.
3. As a **Manager**, I want to register a farm so that it can be officially managed and monitored within the system.
4. As a **Manager**, I want to delete a farm so that outdated or closed farms are no longer stored in the system.

3 Architecture notebook

3.1 Key quality requirements

Performance

The system must provide fast and responsive access to dashboards, allowing users to view animal data with minimal delay (maximum 5 seconds). Since real-time monitoring is critical, the platform should support low-latency data updates and efficiently handle frequent incoming data streams from multiple animals. Dashboard loading times should remain short, even as data volume increases.

Scalability

The platform should be able to scale to support multiple farms, each with several barns and a large number of animals. As the number of monitored metrics and incoming data messages grows, the system must handle increased load without performance degradation.

Reliability & Availability

The system must ensure continuous operation and at least 99% availability, as farmers rely on it for timely insights into animal health. It should be resilient to failures, with mechanisms for fault tolerance, message recovery, and data consistency. In case of component failure, the system should recover quickly without significant data loss.

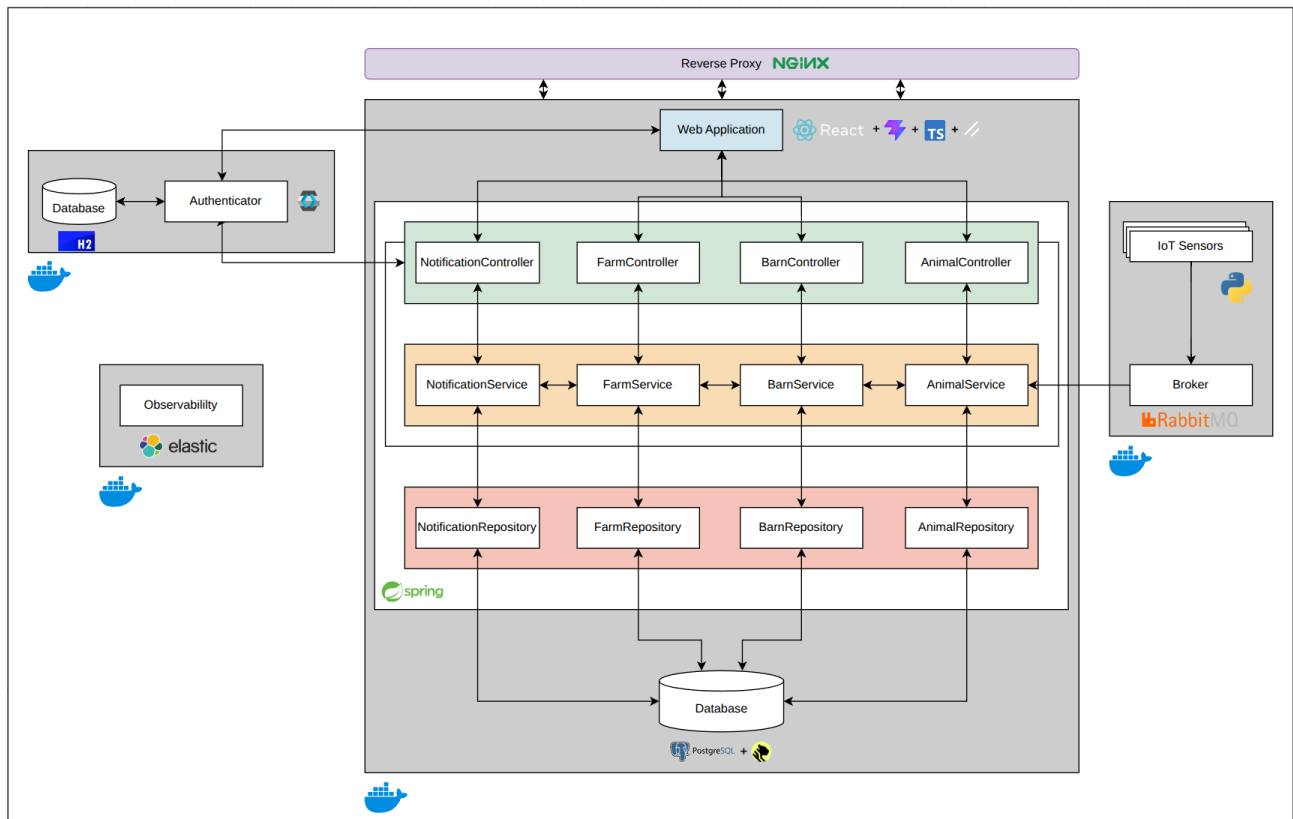
Usability

The platform should provide an intuitive and user-friendly interface, enabling farmers to quickly understand the status of their animals. Dashboards must present complex data in a clear and accessible way, using visualizations and alerts to highlight critical situations. The system should minimize the time and effort required to interpret data and take action.

Portability

The full system must run on any machine with Docker. Every component is containerised and orchestrated with Docker Compose.

3.2 Architecture view



The system follows a layered architecture deployed entirely via Docker containers, with NGINX acting as a reverse proxy that routes all incoming traffic to the appropriate services. The frontend is a React + TypeScript web application that communicates with a Spring Boot backend, which is organized into three distinct layers: a Controller layer (NotificationController, FarmController, BarnController, AnimalController) that handles HTTP request routing, a Service layer that encapsulates business logic with inter-service communication, and a Repository layer that manages data persistence through a PostgreSQL database extended with TimescaleDB for time-series storage. External to the main backend container, an authentication service backed by its own dedicated H2 instance handles identity verification, while a RabbitMQ broker receives telemetry published by IoT sensors simulated via a Python-based Simulator, delivering real-time metric data into the AnimalService for processing. System observability is provided by an Elastic stack instance running in a separate container, enabling centralized logging and monitoring across all services.

3.3 Technology Stack

For the frontend we will be using **React**, **Vite**, **TypeScript**, and for components, we will use **ShadCN**, **Lucide**, and **Recharts**.

To authenticate our users and manage their permission we will use **Keycloak**.

For the backend, we will use **Spring** and structure our project by using **Maven** and **SpringDoc OpenAPI** for API documentation.

Regarding the database, we will be using a **PostgreSQL** with **TimescaleDB** extension.

To handle all the incoming IoT sensor data, we will be using a message broker, specifically, **RabbitMQ**.

The Simulators are generated with the use of **Python** scripts with **Pika** for supporting amqp protocol.

For Observability we have an **ELK** Stack (**Filebeat**, **Logstash**, **Elasticsearch**, **Kibana**).

In the end, all of this will be containerized by the use of **Docker**.

4 Information Model

4.1 Database Entities

Farm

Represents a physical farm owned by a farmer. A farm can contain one or more barns.

Column	Type	Description
id	BIGSERIAL	Primary key identifier for the farm
name	VARCHAR(255)	Display name of the farm
location	VARCHAR(255)	Physical address or region of the farm
zipcode	VARCHAR(20)	Postal or ZIP code of the farm
farmer_id	VARCHAR(255)	Keycloak user UUID representing the farm owner

Barn

Represents a building or enclosure within a farm. A barn houses one or more animals.

Column	Type	Description
id	BIGSERIAL	Primary key identifier for the farm
name	VARCHAR(255)	Display name of the farm
farmer_id	VARCHAR(255)	Foreign key referencing farms(id)

Animal

Represents a livestock animal housed within a barn. Each animal is identified in the simulator using a composite simulator identifier in the format {type}-{id} (e.g., cow-3).

Column	Type	Description
id	BIGSERIAL	Primary key identifier for the animal
name	VARCHAR(255)	Name assigned to the animal
type	VARCHAR(50)	Species type (cow, sheep, pork)
weight	NUMERIC(6,2)	Initial weight in kilograms
height	NUMERIC(6,2)	Height in centimeters
notes	TEXT	Veterinary or farmer observations
vet_email	VARCHAR(255)	Veterinarian contact email
vet_phone	VARCHAR(50)	Veterinarian contact phone number
barn_id	BIGINT	Foreign key referencing barns(id)

Simulator Identifier

Animals are referenced in the simulator using : {type}-{id}

Example: cow-3, sheep-12

AnimalMetric

A TimescaleDB hypertable used to store time-series sensor readings for animals. Each row records one or more biometric or behavioral metrics captured at a specific timestamp. The composite primary key is: (time, animal_id)

Column	Type	Description
time	TIMESTAMPTZ	Timestamp of the recorded measurement

animal_id	VARCHAR(50)	Simulator animal identifier (e.g., cow-3)
heart_rate	INT	Heart rate in beats per minute (BPM)
temperature	NUMERIC(5,2)	Body temperature in °C
stress	NUMERIC(4,1)	Stress index ranging from 0–10
movement	INT	Activity or movement count during the sampling interval

AnimalWeight

A separate TimescaleDB hypertable used to store animal weight readings. Weight measurements are sampled less frequently than other biometric metrics and are therefore separated from the main metrics table. The composite primary key is: (time, animal_id)

Column	Type	Description
time	TIMESTAMPTZ	Timestamp of the recorded weight measurement
animal_id	VARCHAR(50)	Simulator animal identifier (e.g., cow-3)
weight	NUMERIC(6,1)	Animal body weight in kilograms

4.2 API Documentation

Farmers API (manager role)

- **POST /api/farmers:** Creates a new farmer account in Keycloak and assigns the farmer role.
- **GET /api/farmers:** Returns all farmers and their associated farms.
- **PUT /api/farmers/{id}:** Updates farmer information using the Keycloak user ID.
- **DELETE /api/farmers/{id}:** Deletes a farmer account from Keycloak.
- **GET /api/farmers/{farmerId}/farms:** Returns all farms belonging to a specific farmer.

Farms API (manager write, farmer read)

- **POST /api/farms:** Creates a new farm associated with a farmer.
- **PUT /api/farms/{id}:** Updates an existing farm.
- **GET /api/farms/me:** Returns farms associated with the authenticated farmer.
- **GET /api/farms/farmer/{farmerId}:** Returns farms belonging to a specific farmer.

Barns API (farmer role)

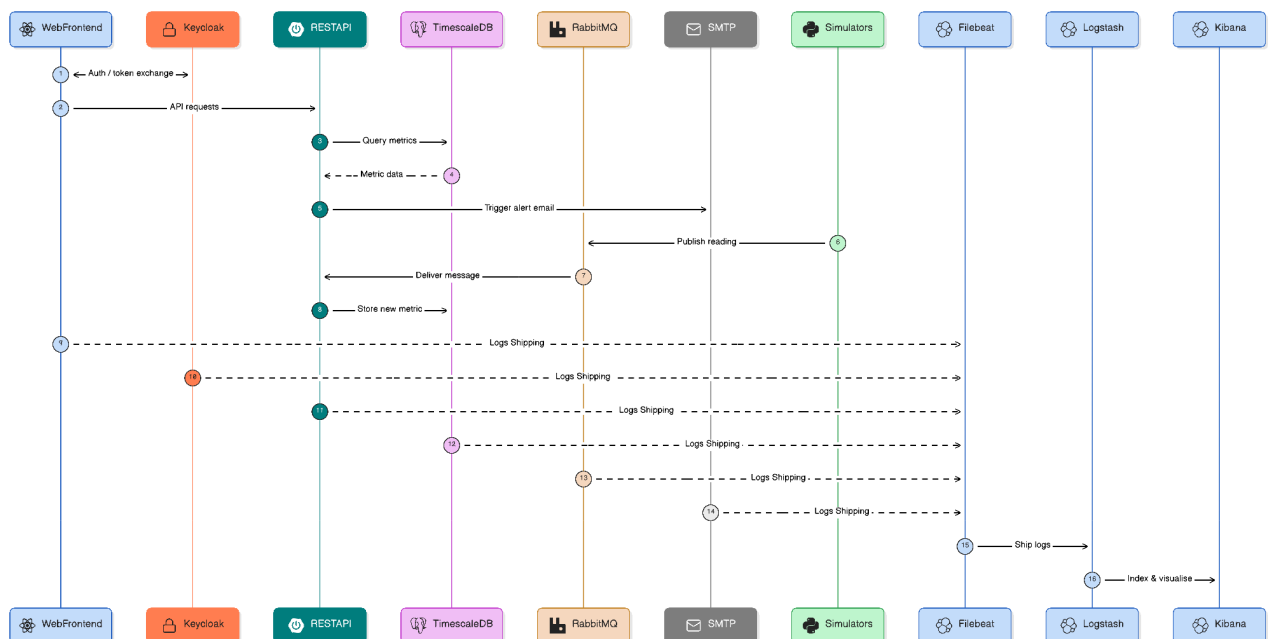
- **GET /api/barns?farmId={id}:** Returns all barns, optionally filtered by farm ID.

- **GET /api/barns/{id}**: Returns a specific barn by ID.
- **POST /api/barns**: Creates a new barn.
- **PUT /api/barns/{id}**: Updates an existing barn.
- **DELETE /api/barns/{id}**: Deletes a barn.
- **GET /api/barns/{id}/animals**: Returns all animals within a barn.

Animals API (farmer role)

- **POST /api/animals**: Adds a new animal to a barn.
- **DELETE /api/animals/{id}**: Removes an animal using its numeric database ID.
- **GET /api/animals/all**: Returns all animals in the system. Used by simulators.
- **GET /api/animals/{id}/metrics/heart-rate**: Returns the latest heart rate measurement.
- **GET /api/animals/{id}/metrics/temperature**: Returns the latest body temperature measurement.
- **GET /api/animals/{id}/metrics/stress**: Returns the latest stress measurement.
- **GET /api/animals/{id}/metrics/movement**: Returns the latest movement count.
- **GET /api/animals/{id}/metrics/movement/history?from=&to=**: Returns daily movement aggregates for a given time range.
- **GET /api/animals/{id}/metrics/weight/history?from=&to=**: Returns weekly average weight values for a given time range.
- **GET /api/animals/{id}/notes**: Returns notes associated with the animal.
- **PUT /api/animals/{id}/notes**: Updates notes associated with the animal.
- **GET /api/animals/{id}/vet**: Returns veterinarian contact information.
- **PUT /api/animals/{id}/vet**: Updates veterinarian contact information.
- **POST /api/animals/{id}/notify-vet**: Send a veterinarian email containing a PDF health report.

4.3 Module Interactions



The system interactions can be summarized as follows:

- The process starts when the **WebFrontend** authenticates the user through **Keycloak**, which returns an access token. The frontend then uses this token to send API requests to the **REST API**.
- The **REST API** interacts with **TimescaleDB** to query and retrieve stored metrics, and also stores new metric data received from other components.
- Simulators continuously generate metric data and publish it to **RabbitMQ**, which acts as a message broker. The **REST API** consumes these messages from **RabbitMQ** and stores the data in **TimescaleDB**.
- When conditions are met, the **REST API** sends alert notifications through **SMTP**.
- In parallel, all components send logs to **Filebeat**, which forwards them to **Logstash**. **Logstash** processes the logs and sends them to **Kibana**, where they are indexed and visualized.

5 Data Simulation

The AgriSense data simulator is a Python-based component that generates realistic sensor readings for registered animals and publishes them to RabbitMQ for downstream processing by the API.

5.1 Architecture Overview

The simulator is composed of a shared base module and five independent sensor simulators, each running as a separate Docker container. All containers are built from the same image; the **SIMULATOR** environment variable selects which script to execute at startup.

```

simulator/
├── base.py          # Shared utilities and animal sync logic
├── sim_heart_rate.py # Heart rate sensor simulator
├── sim_temperature.py # Body temperature sensor simulator
├── sim_stress.py    # Stress level sensor simulator
├── sim_movement.py  # Movement activity sensor simulator
├── sim_weight.py    # Weight sensor simulator
├── requirements.txt # Python dependencies (pika 1.3.2)
└── Dockerfile

```

5.2 Base Module (base.py)

The base module provides the shared infrastructure used by all simulators:

- **Animal Registry Sync:** The `sync_animals()` function polls the REST API (**GET /api/animals/all**) every **POLL_INTERVAL** seconds (default: 30 s) to retrieve the current list of registered animals. It dynamically starts a new simulation thread for each newly registered animal and stops threads for animals that have been removed. This means the simulator automatically adapts to changes in the animal registry without requiring a restart.
- **RabbitMQ Connection:** The `connect()` function establishes a durable connection to RabbitMQ with automatic retry logic, waiting 3 seconds between attempts until the broker is available. Messages are published with **delivery_mode=2** (persistent), ensuring they survive broker restarts.

- Species Profiles: Each supported species (cow, sheep, pork) has a dedicated profile defining the physiological parameters used to generate realistic readings:

Parameter	Description
base	Baseline value for the metric
amp	Amplitude of the sinusoidal variation
freq	Frequency of the sinusoidal cycle
noise	Maximum random noise added per reading

If an animal's species is not recognised, the cow profile is used as the default.

5.3 Sensor Simulators

Each simulator runs an independent loop per animal, generating values and publishing them to a dedicated RabbitMQ queue.

- **Heart Rate (sim_heart_rate.py):** Queue **animal.heart_rate**
Generates heart rate values in bpm using a sinusoidal function with added Gaussian noise. Publishes every 3 seconds. Formula: $\text{base} + \text{amp} \times \sin(t \times \text{freq}) + \text{noise}$
- **Body Temperature (sim_temperature.py):** Queue **animal.temperature**
Generates body temperature values in °C using the same sinusoidal model. Publishes every 3 seconds.
- **Stress Level (sim_stress.py):** Queue **animal.stress**
Generates a stress index on a scale of 0.0 to 10.0, clamped to that range. Publishes every 3 seconds.
- **Movement (sim_movement.py):** Queue **animal.movement**
Generates a random integer activity count within species-specific min/max bounds. Publishes every 3 seconds.
- **Weight (sim_weight.py):** Queue **animal.weight**
Generates weight values in kg with small random noise around a species baseline. Publishes every 60 seconds, reflecting the slower rate at which weight changes in real animals.

5.4 Message Format

All simulators publish JSON messages to their respective queues with the following common structure:

```
{
  "animalId": "<simulatorId or type-id>",
  "timestamp": 1748367033.792,
  "<metric>": <value>
}
```

The metric field name matches the sensor type (e.g., heartRate, temperature, stressLevel, movement, weight).

5.5 Deployment

Each simulator runs as a separate Docker Compose service (**sim-heart-rate**, **sim-temperature**, **sim-stress**, **sim-movement**, **sim-weight**). All services depend on RabbitMQ being healthy and the API being started before launching.

The shared Docker image is built from **simulator/Dockerfile** using Python 3.11 slim.

6 Observability (ELK)

AgriSense uses an ELK stack (Elasticsearch, Logstash, Kibana) together with Filebeat to provide centralized log collection, processing, and visualization across all services. The entire stack runs as Docker containers alongside the rest of the application. Once the stack is running, Kibana is available at <http://localhost:5601>.

6.1 Architecture

The log pipeline flows in one direction: Docker containers → Filebeat → Logstash → Elasticsearch → Kibana.

Filebeat acts as the log shipper. It runs with access to the host Docker socket and the container log directory, collecting stdout/stderr from every running container. It attaches Docker metadata (container name, image, labels) to each event and forwards everything to Logstash on port 5044.

Logstash receives events from Filebeat and applies a three-branch parsing pipeline depending on the log format:

- **API logs:** Spring Boot emits structured JSON via LogstashEncoder. Logstash detects messages starting with "{", parses the JSON, renames the level and logger_name fields, and tags the event with service_type: api.
- **Simulator logs:** Python simulators emit lines in the format [metric] simulatorId: value unit. Logstash uses a grok pattern to extract the metric name, simulator ID, numeric value, and unit, tagging with service_type: simulator.
- **Infrastructure logs:** Elasticsearch, Kibana, RabbitMQ, and similar services emit lines in the format [TIMESTAMP][LEVEL][logger] message. Logstash parses these with grok and tags them with service_type: infrastructure.

Internal ELK containers are dropped entirely to avoid self-monitoring noise.

After branch-specific parsing, all events go through a shared post-processing step: log levels are uppercased and normalized (WARNING → WARN, NOTICE → INFO, VERBOSE → DEBUG), a service_name field is set from the Docker container name, and internal metadata fields are removed.

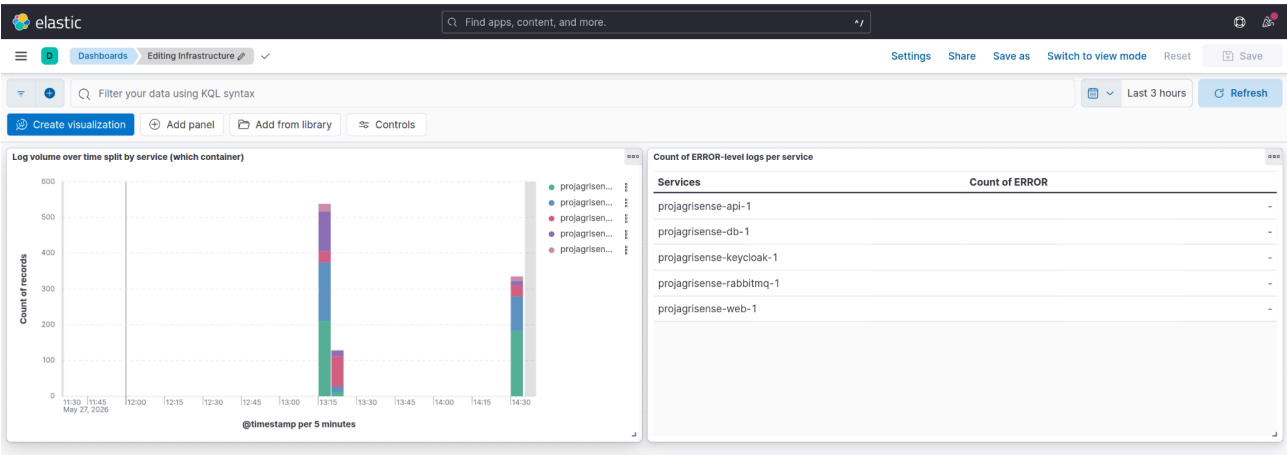
Elasticsearch stores all processed events in daily rolling indices named agrisense-YYYY.MM.dd. It runs as a single-node cluster with security disabled, suitable for development.

Kibana is auto-configured on first startup. A dedicated setup container imports a saved objects file via the Kibana API, which provisions the agrisense-* index pattern and three pre-built dashboards.

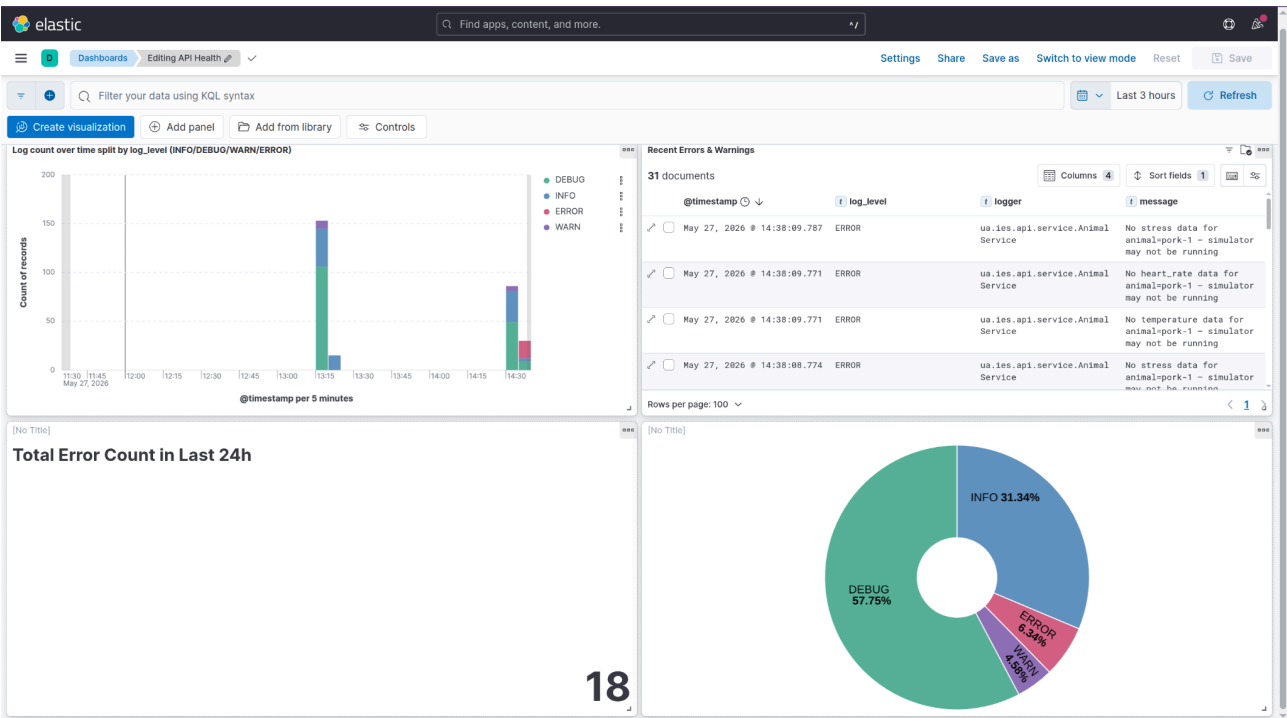
6.2 Dashboards

Three dashboards are available in Kibana under Analytics → Dashboards:

- Infrastructure dashboard: provides a bird's-eye view of all containers, showing log volume over time split by service and a table ranking containers by error count.

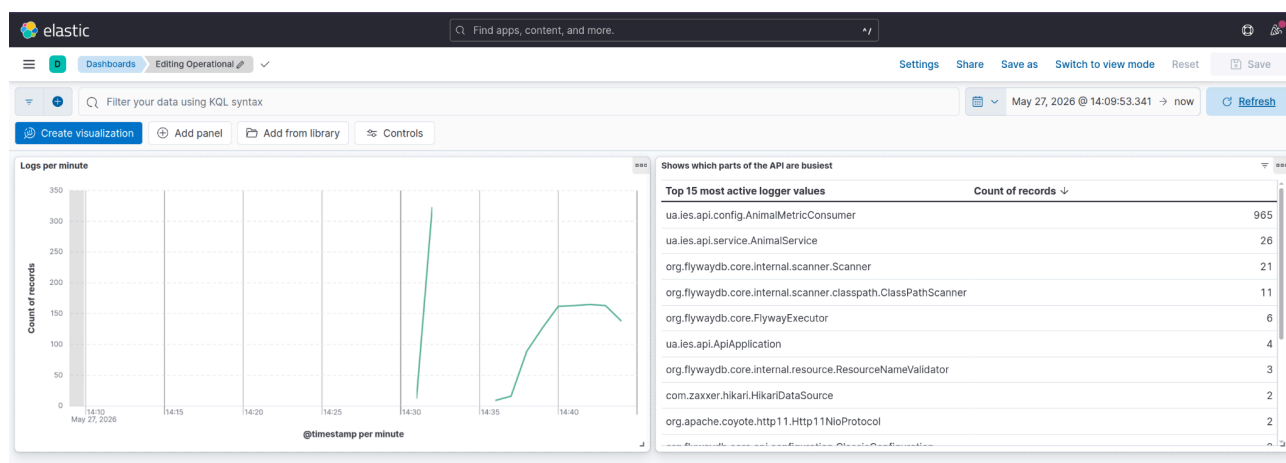


- API Health dashboard: focused on the Spring Boot service, with log count over time split by log level, a live table of recent errors and warnings, a log level distribution donut chart, and a total error count metric for the last 24 hours.



18

- Operational dashboard: focused on API activity, showing logs per minute as a line chart and a table of the top 15 most active Java logger classes.



7 Deployment

Our app is containerized with the use of Docker, all services are started automatically in dependency order. The API service waits for Keycloak, RabbitMQ, and TimescaleDB health checks to pass before starting. The simulator services start only after the API becomes available.

After startup, the main services are accessible through the following URLs:

- **Web frontend:** <http://localhost:3000>
- **REST API:** <http://localhost:8080>
- **Swagger UI:** <http://localhost:8080/swagger-ui.html>
- **Keycloak administration console:** <http://localhost:8180>
- **RabbitMQ management interface:** <http://localhost:15672>
- **Kibana dashboard:** <http://localhost:5601>

The Keycloak realm named agrisense is automatically imported during the first startup using the configuration file located at `projAgrisense/keycloak/agrisense-realm.json`.

Database schema migrations are managed using Flyway. On API startup, Flyway automatically applies any pending migrations located in `api/src/main/resources/db/migration/`.

We also use GitHub Actions for continuous integration. The workflow is defined in `.github/workflows/ci.yml` and is triggered on every push or pull request targeting the main or dev branches.

The current pipeline executes a **docker compose build** command inside the **projAgrisense/** directory to verify that all services build successfully. This includes all the services.

If any build step fails, merges to the protected branches are blocked until the issue is resolved.

8 Future Implementations and Enhancements

This section identifies the gaps that must be addressed before AgriSense can be deployed to a production environment, as well as enhancements that would improve the system's robustness, usability, and scalability.

8.1 Production Deployment Requirements

8.1.1 Environment Configuration and Secrets Management

The current configuration relies on hardcoded default values and plain-text credentials scattered across **application.properties**, **docker-compose.yml**, and the frontend source code. Before any production deployment, the following must be addressed:

- All secrets (database password, Keycloak admin credentials, SMTP password, RabbitMQ credentials) must be moved out of source code and managed through a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault, or Docker Secrets).
- The SMTP password (**mbnvohocbkctrqqb**) is currently hardcoded in **application.properties** and must be rotated and injected via environment variable.
- The frontend has the Keycloak URL and API base URL hardcoded to **localhost** in **keycloak.ts** and **api.ts**. These must be made configurable at build time (e.g., via Vite environment variables) to support different deployment targets.

8.1.2 Data Retention and TimescaleDB Policies

The **animal_metric** and **animal_weight** tables are TimescaleDB hypertables but have no retention policies defined. At a publishing rate of one reading every 3 seconds per animal, data volume grows rapidly. Before production deployment, automated data retention policies (e.g., drop data older than 90 days) and continuous aggregates for long-term trend queries should be configured directly in TimescaleDB.

8.1.3 Container Orchestration

The current setup uses Docker Compose, which is suitable for development but not for production workloads. A migration to Kubernetes or a managed container service (e.g., AWS ECS, Google Cloud Run) would provide:

- Automatic restarts and health-based scheduling.
- Horizontal scaling of the API and simulator containers.
- Rolling deployments without downtime.

8.2 Functional Enhancements

8.2.1 Automated Alerting

The system currently supports manual vet notification triggered by the farmer through the UI. An automated alerting layer should be added to detect anomalous readings and trigger notifications without human intervention. This would involve defining per-species threshold rules (e.g., heart rate above 120 bpm, temperature above 40.5 °C, stress above 8.0) and evaluating incoming metrics against them in the **AnimalMetricConsumer**. Alerts could be delivered via the existing email infrastructure or through additional channels (SMS, push notifications).

8.2.2 Real Sensor Integration

The simulator currently generates synthetic data using sinusoidal functions with noise. The architecture (RabbitMQ queues, species profiles, per-animal threads) is already well-suited for replacement with real IoT sensor data. The required work is:

- - Replacing or supplementing the simulator containers with MQTT or AMQP bridge adapters that receive data from physical sensors and publish to the same RabbitMQ queues.
- Extending the species profiles and metric ranges to reflect calibrated real-world values.
- Adding support for additional animal species beyond the current three (cow, sheep, pork).

8.2.3 Live Metric Streaming

The dashboard currently polls the API for the latest metric values. For a more responsive user experience, the API should expose a WebSocket or Server-Sent Events (SSE) endpoint that pushes new readings to connected clients as they arrive from RabbitMQ, eliminating the need for polling and reducing perceived latency.

8.2.4 Role-Based Access Control Refinement

The current security model grants all authenticated users the **farmer** role with uniform access to all resources. A multi-tenant deployment requires:

- Scoping data access so that a farmer can only read and manage animals belonging to their own farms.
- Adding a veterinarian role with read-only access to animal health data.
- Adding an administrator role for user and farm management.

8.2.5 Audit Logging

There is currently no audit trail for sensitive operations such as animal creation and deletion, vet contact updates, or vet notifications sent. An audit log (stored in the database or forwarded to Elasticsearch) would be required for regulatory compliance in agricultural contexts and for diagnosing issues in production.

8.2.6 Mobile Application

The frontend is responsive and includes a mobile layout (**useIsMobile** hook, **DashboardTable** fallback), but it is delivered as a web application. A native or hybrid mobile application would improve usability for farmers working in the field, enabling offline access to the last known readings and push notification support for alerts.